Primo 1: El estadio de barrio

Concepto: estructura de una clase con atributos, métodos y constructor.

Enunciado: escribe un programa Java. Crea la clase `Estadio`, sin `public` y declarada antes de `Main`, con estos atributos sin modificador de acceso: `nombre` (`String`), `aforo` (`int`) y `vendidas` (`int`, las entradas vendidas). Estos son sus miembros:

- Un constructor `Estadio(String nombre, int aforo)` que guarda ambos valores e inicializa `vendidas` a 0.
- El método `void vender()`, que suma 1 a `vendidas`.
- El método `String ficha()`, que devuelve el texto `NOMBRE (aforo A) - vendidas: V`, con los valores de los atributos.

En `main` crea dos objetos con `new`: `e1` con `"Campo del Garbanzo"` y aforo 500, y `e2` con `"Estadio Tortilla"` y aforo 1200. Llama tres veces a `e1.vender()` y ninguna a `e2.vender()`. Después imprime `e1.ficha()` y `e2.ficha()`, en ese orden.

Salida prevista:

text

```
Campo del Garbanzo (aforo 500) - vendidas: 3
Estadio Tortilla (aforo 1200) - vendidas: 0
```

PROMETEO
by thePower

Primo 2: La caja de la peña

Concepto: visibilidad `private`/`public`, getters y encapsulación con validación.

Enunciado: escribe un programa Java. Crea la clase `PenaFutbol`, sin `public` y declarada antes de `Main`, con los atributos **privados** `nombre` (`String`) y `euros` (`int`, empieza en 0). Estos son sus miembros:

- Un constructor `PenaFutbol(String nombre)`.
- Los getters públicos `String getNombre()` e `int getEuros()`.
- El método público `boolean ingresar(int euros)`. Solo acepta cuotas de 5, 10 o 20 euros. Si recibe otro valor, devuelve `false` sin modificar nada. Si el valor es válido, lo suma a la caja y devuelve `true`.

En `main` crea la peña con el nombre `"Peña Los Pellizcos"` y recorre con un `for` el array `{10, 5, 7, 20, 10}`. Para cada cuota (la jornada se numera desde 1):

- Si el ingreso se acepta, imprime `Cuota N: +E euros, total T`.

- Si se rechaza, imprime `Cuota N: billete falso (E euros), se ignora`.

Al terminar imprime `Caja: NOMBRE con T euros`, usando los getters. Intentar acceder desde `Main` directamente al atributo `euros` no compilaría, y no debes hacerlo.

Salida prevista:

text

```
Cuota 1: +10 euros, total 10
Cuota 2: +5 euros, total 15
Cuota 3: billete falso (7 euros), se ignora
Cuota 4: +20 euros, total 35
Cuota 5: +10 euros, total 45
Caja: Peña Los Pellizcos con 45 euros
```

Primo 3: La ronda de penaltis

Concepto: atributos y métodos `static` (contadores compartidos).

Enunciado: escribe un programa Java. Crea la clase `Tanda`, sin `public` y declarada antes de `Main`, que representa una tanda de penaltis. Tiene dos atributos **estáticos** de tipo `int`, `penaltisMarcados` y `tandasJugadas`, ambos a 0. Tiene también estos atributos de instancia: `equipoA` y `equipoB` (`String`) y `penaltisA` y `penaltisB` (`int`). Estos son sus miembros:

- Un constructor `Tanda(String equipoA, int penaltisA, String equipoB, int penaltisB)` que guarda los datos, suma `penaltisA + penaltisB` a `penaltisMarcados` y suma 1 a `tandasJugadas`.
- El método `String resultado()`, que devuelve `EQUIPOA PA-PB EQUIPOB`.
- El método **estático** `String resumen()`, que devuelve `Tandas: T, marcados: M, media entera: E`. `E` es `penaltisMarcados / tandasJugadas` con división entera, o 0 si `tandasJugadas` vale 0.

En `main`, imprime primero `Antes de empezar ->` seguido de `Tanda.resumen()`. Crea después estas tres tandas, en este orden: `"Cebolla FC"` 4 y `"Pimiento CF"` 3, `"Real Berenjena"` 5 y `"Calabacín United"` 5, `"Deportivo Lenteja"` 0 y `"Deportivo Alubia"` 0. Imprime el resultado de cada una y termina con `Tras la ronda ->` seguido de `Tanda.resumen()`.

Salida prevista:

text

```
Antes de empezar -> Tandas: 0, marcados: 0, media entera: 0
Cebolla FC 4-3 Pimiento CF
Real Berenjena 5-5 Calabacín United
```

```
Deportivo Lenteja 0-0 Deportivo Alubia
Tras la ronda -> Tandas: 3, marcados: 17, media entera: 5
```

Primo 4: La tienda del club

Concepto: array de objetos de clases propias, colaboración entre clases y paquetes (simulados con comentarios).

Enunciado: escribe un programa Java con tres clases en un único archivo. Indica en comentarios los paquetes que tendrían en un proyecto real: `futbol.modelo` para `Producto`, `futbol.tienda` para `Tienda` y `futbol.app` para `Main`.

- La clase `Producto` tiene los atributos privados `nombre` (`String`) y `precio` (`int`, euros). Tiene un constructor `Producto(String nombre, int precio)` y los getters públicos `getNombre()` y `getPrecio()`.
- La clase `Tienda` tiene un array privado `Producto[] vitrina` de capacidad 3 (`new Producto[3]`) y un contador privado `numero`, que empieza en 0. Tiene el método `boolean exponer(Producto p)`, que guarda `p` en el primer hueco libre y devuelve `true`. Si la vitrina está llena, devuelve `false`. Tiene `int valorTotal()`, que suma los precios de los productos expuestos recorriendo **solo** hasta `numero`, y `int getNumero()`.
- En `main`, crea una `Tienda` y un array de cuatro productos: `"Bufanda"` (15), `"Gorra"` (12), `"Balón firmado"` (40) y `"Llavero"` (3). Recórrelo con un `for`. Si el producto se expone, imprime `Expuesto: NOMBRE (P euros)`. Si no cabe, imprime `Vitrina llena: no cabe NOMBRE`. Al final imprime `Valor total: V euros en N productos`.

Salida prevista:

text

```
Expuesto: Bufanda (15 euros)
Expuesto: Gorra (12 euros)
Expuesto: Balón firmado (40 euros)
Vitrina llena: no cabe Llavero
Valor total: 67 euros en 3 productos
```

Primo 5: El delantero

Concepto: herencia básica (`extends`, `super(...)`, `@Override`).

Enunciado: escribe un programa Java con tres clases en un único archivo. Solo se pide herencia básica, sin clases abstractas ni interfaces.

- La clase padre `Futbolista` tiene los atributos privados `nombre` (`String`) y `dorsal` (`int`). Su constructor es `Futbolista(String nombre, int dorsal)`. El método `String descripcion()` devuelve `#DORSAL NOMBRE`.
- La clase `Delantero` hereda de `Futbolista` con `extends`. Añade el atributo privado `goles` (`int`, empieza en 0). Su constructor `Delantero(String nombre, int dorsal)` llama a `super(nombre, dorsal)` en su primera línea. Añade el método `void marcar()`, que suma 1 a `goles`. Sobrescribe `descripcion()` con `@Override` para que devuelva `super.descripcion()` más `- goles: N`.
- En `main` crea estos tres objetos: un `Futbolista` llamado `"Lola Lateral"` con el dorsal 4, un `Delantero` llamado `"Rafa Rematador"` con el dorsal 11 y un `Delantero` llamado `"Suplente Soñoliento"` con el dorsal 20. Llama cuatro veces a `marcar()` sobre Rafa y ninguna sobre el suplente. Imprime la descripción de los tres, en el orden indicado.

Salida prevista:

text

```
#4 Lola Lateral
#11 Rafa Rematador - goles: 4
#20 Suplente Soñoliento - goles: 0
```

Tabla de primos

Primo	Hermano de	Qué cambia	Qué se mantiene
1. Estadio de barrio	1. Ficha de jugador	Clase `Estadio`, atributos (`aforo`, `vendidas`) y datos (3 entradas y 0)	Constructor con `this`, método que modifica el estado, método que devuelve texto y objetos independientes
2. Caja de la peña	2. Puntos del equipo	Clase `PenaFutbol`, valores válidos (5, 10, 20), datos del array y mensajes	Atributos `private`, getters, método `boolean` con validación y salida con el rechazo en la 3.ª posición
3. Ronda de penaltis	3. Estadísticas de la jornada	Clase `Tanda`, nombres de contadores y datos (17 penaltis, media 5 con 17/3)	Dos atributos `static`, constructor que los actualiza, `resumen()` estático y comprobación de 0 antes de dividir

4. Tienda del club	4. Plantilla de fichajes	Clases `Producto` y `Tienda`, datos (4 candidatos, total 67 euros) y textos	Array de objetos con capacidad 3, método `boolean`, suma hasta `numero` y paquetes en comentarios
5. El delantero	5. El portero	Clases `Futbolista` y `Delantero`, atributo `goles`, datos (4 goles y 0)	`extends`, `super(...)` en la primera línea, `@Override` y `super.descripcion()`

PROMETEO
by thePower